



4106 Deep Reinforcement Learning

Module 4: Deep Q Networks



Course Plan

Module Titles
Module 1 – Introduction to Deep Reinforcement Learning
Module 2 – Dynamic Programming and Value Functions
Module 3 – Value Function Approximation and Feature Engineering
Current Focus: Module 4 –Deep Q Networks
Module 5 – Policy Gradient and Actor-Critic Methods
Module 6 – Advanced DRL
Module 7 – Curiosity and Prediction
Module 8 – Multi-agent DRL
Module 9 – Attention and Relational Models
Module 10 – Recent Advances in DRL
Module 11 – Presentations (no content)
Module 12 – Presentations (no content)



Learning Outcomes for this Module

- Recognize the challenges involved in combining deep learning with reinforcement learning
- Describe strategies for mitigating the issues
- Apply integrated planning and learning strategies



Topics for this Module

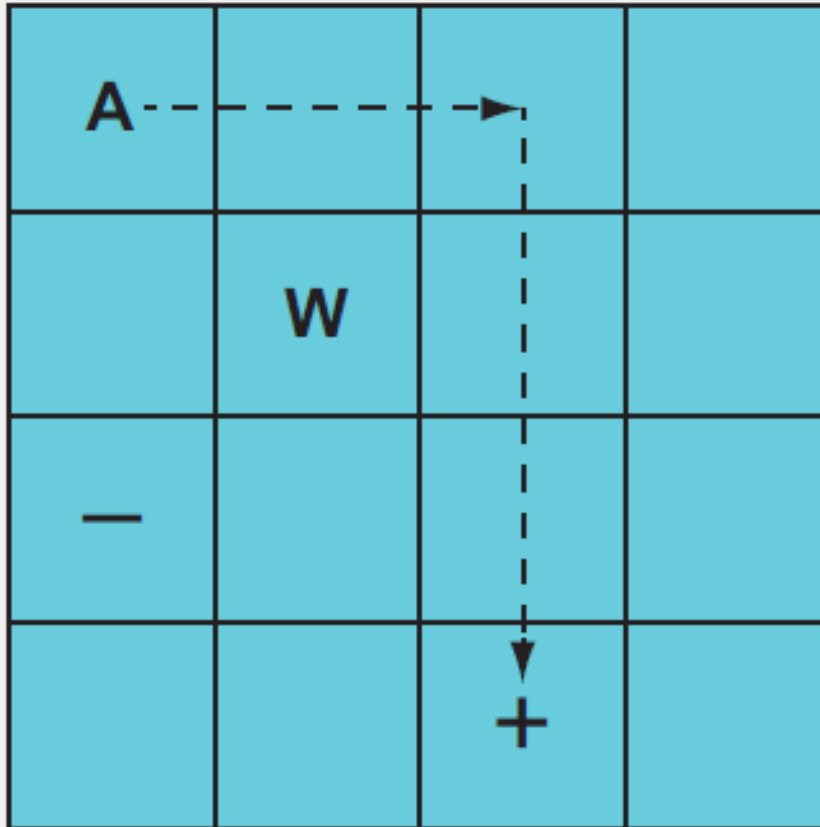
- 4.1 Gridworlds
- 4.2 Deep Q Networks
- 4.3 Integrated Planning and Learning
- 4.4 Resources and Wrap-up



Module 4 – Section 1

Gridworlds

A Simple Gridworld Example



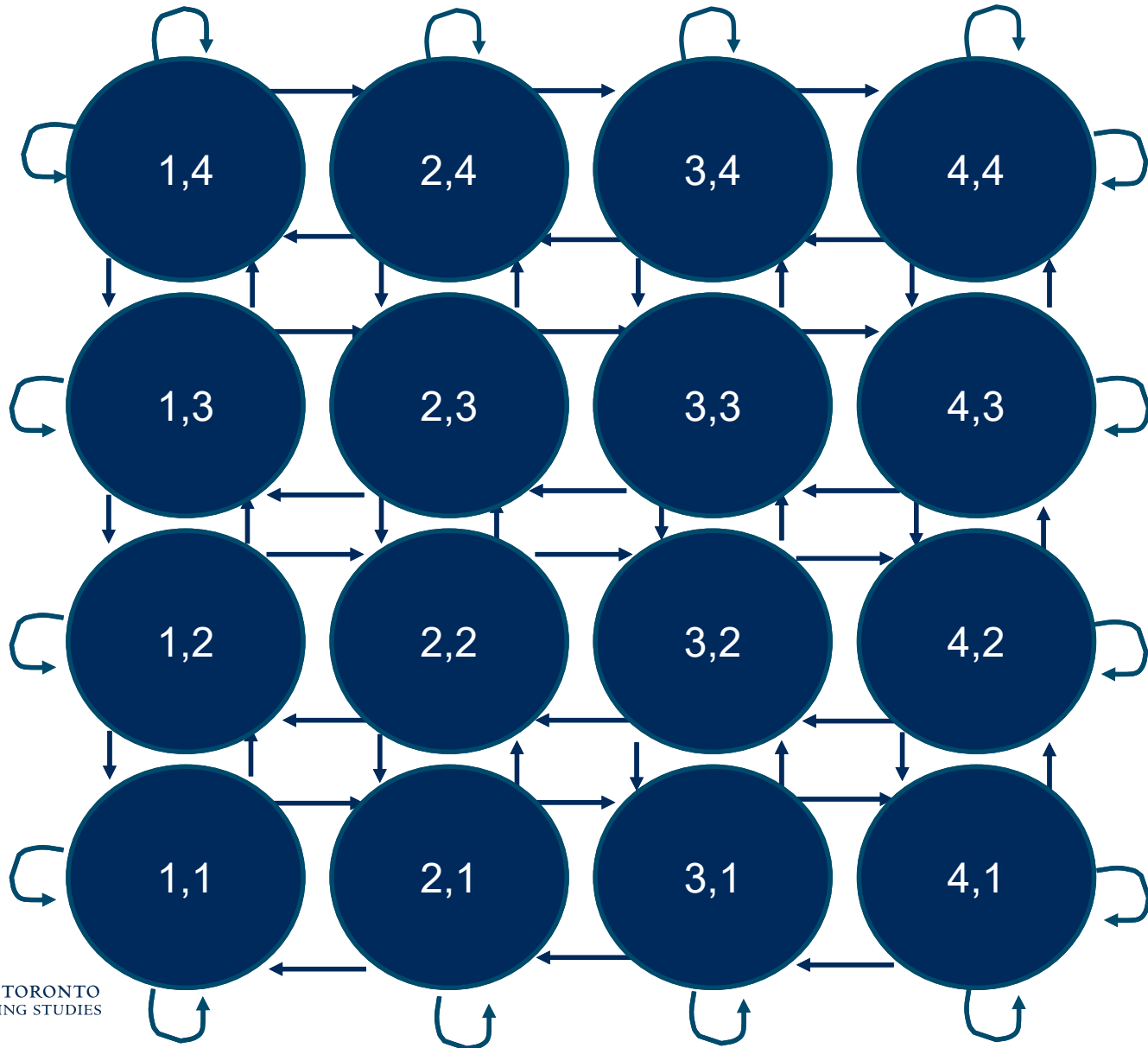
A : Agent

W : Wall

— : Pit

+ : Goal

The GridWorld as a State Diagram



Review of the Four Elements

- **Policy**
 - $\pi(a|s)$
- **Reward Signal**
 - R
- **Value Function**
 - Value of a state: $v_{\pi}(s) = E_{\pi}(G_t | S_t = s)$
 - Value of a state-action: $q_{\pi}(s, a) = E_{\pi}(G_t | S_t = s, A_t = a)$
- **Model of the Environment**
 - $p(s', r | s, a) = \Pr(S_t = s', R_t = r | S_{t-1} = s, A_{t-1} = a)$

Environment Model p as a Table

State	Take Action	New State	Reward	Probability	Terminated
1,1	Up	1,2	-1	1.0	True
1,1	Left	1,1	0	1.0	False
1,1	Down	1,1	0	1.0	False
1,1	Right	2,1	0	1.0	False
2,1	Up	2,2	0	1.0	False
2,1	Left	1,1	0	1.0	False
2,1	Down	2,1	0	1.0	False
2,1	Right	3,1	+1	1.0	True
etc.					

q as a Table

State	Take Action	q
1,1	Up	-1.0
1,1	Left	0.7
1,1	Down	0.7
1,1	Right	0.9
2,1	Up	0.8
2,1	Left	0.8
2,1	Down	0.8
2,1	Right	1.0
etc.		



Module 4 – Section 2

Deep Q Networks

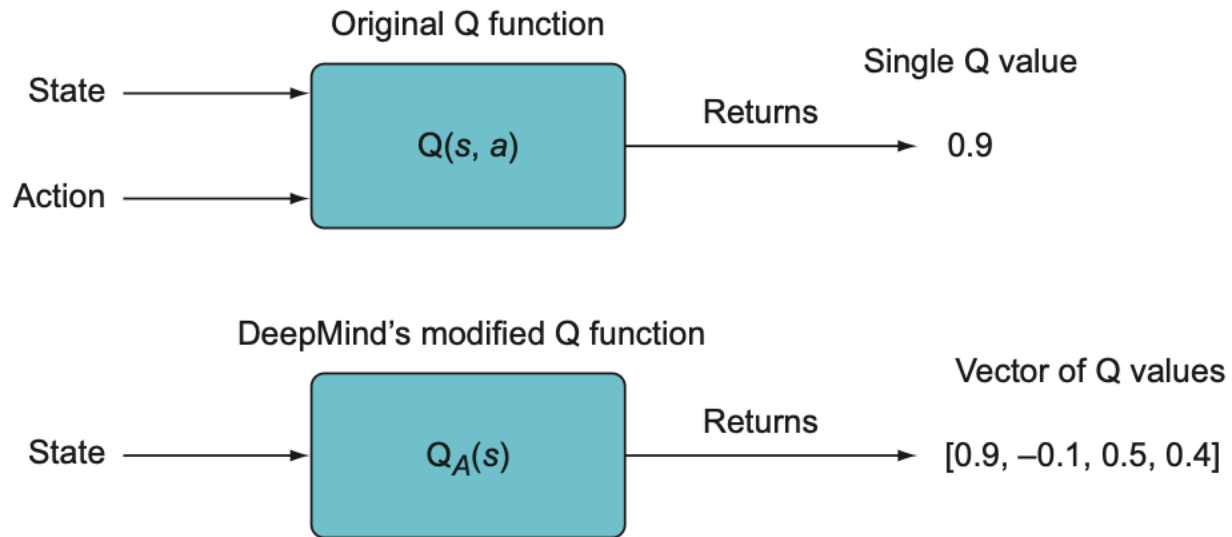
From Tabular to Deep RL

- Replace the discrete states and probability tables with features and approximators
- Now we can have continuous features
- We can learn the value functions using ML techniques
- We can scale up!
- What could possibly go wrong?

The Deadly Triad

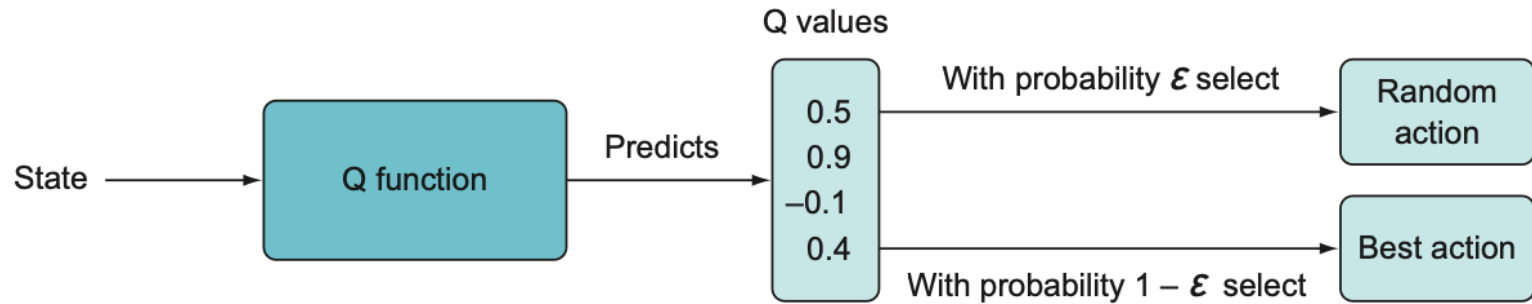
- **Function Approximation:** Using ML models to approximate the state space
 - **Bootstrapping:** Using Temporal Difference (rather than Monte Carlo)
 - **Off-policy Training:** Training the model by following trajectories through the state space other than what the current policy specifies
-
- If you have all three learning is not guaranteed to converge

Predicting Q



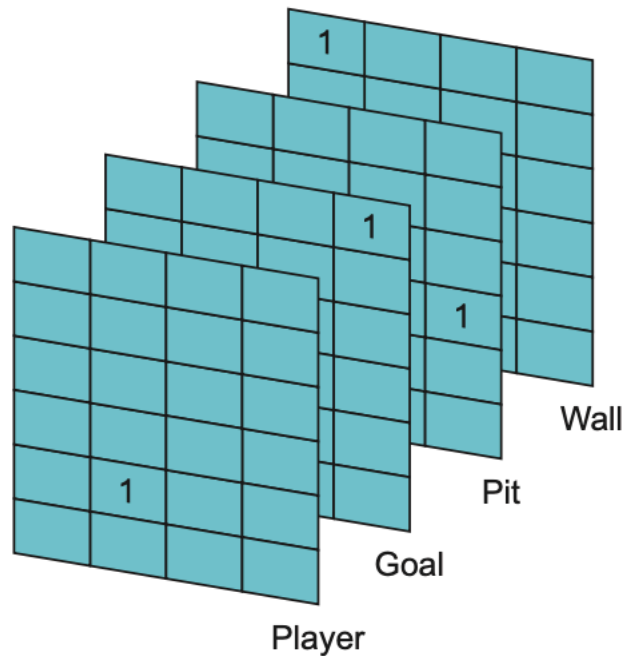
Source: Zai & Brown

Implementing an ϵ -greedy Policy



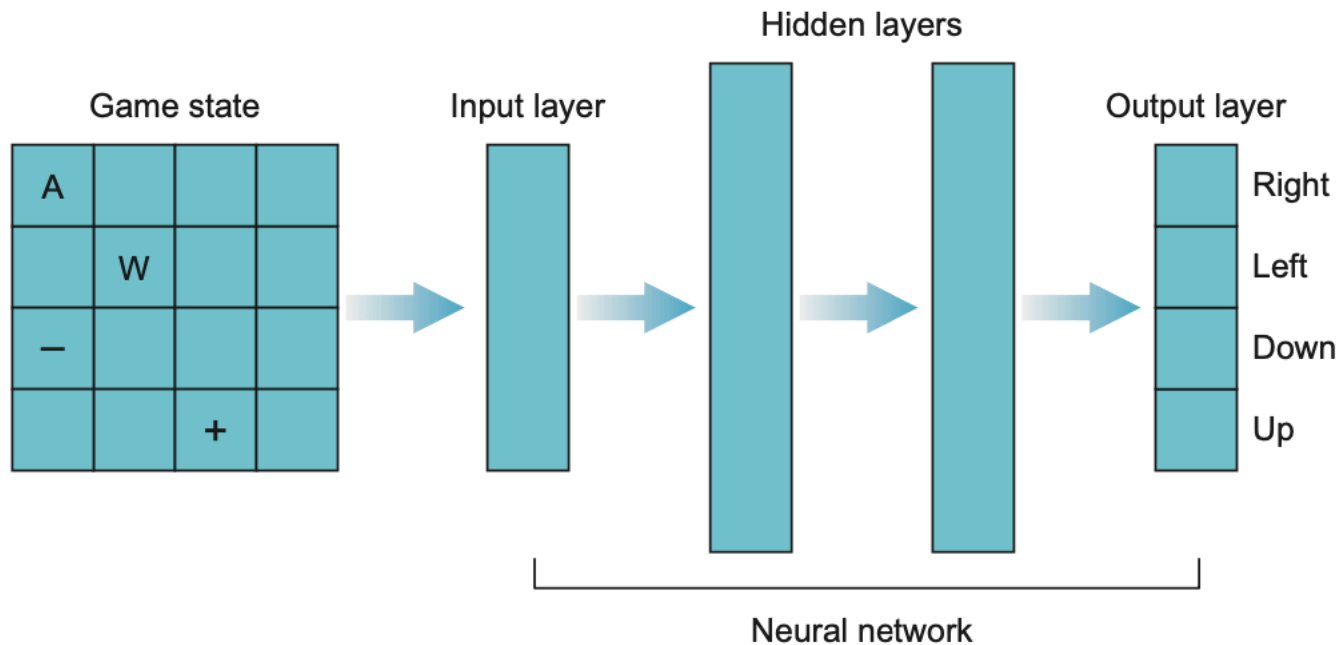
Source: Zai & Brown

Representing the Environment as Features



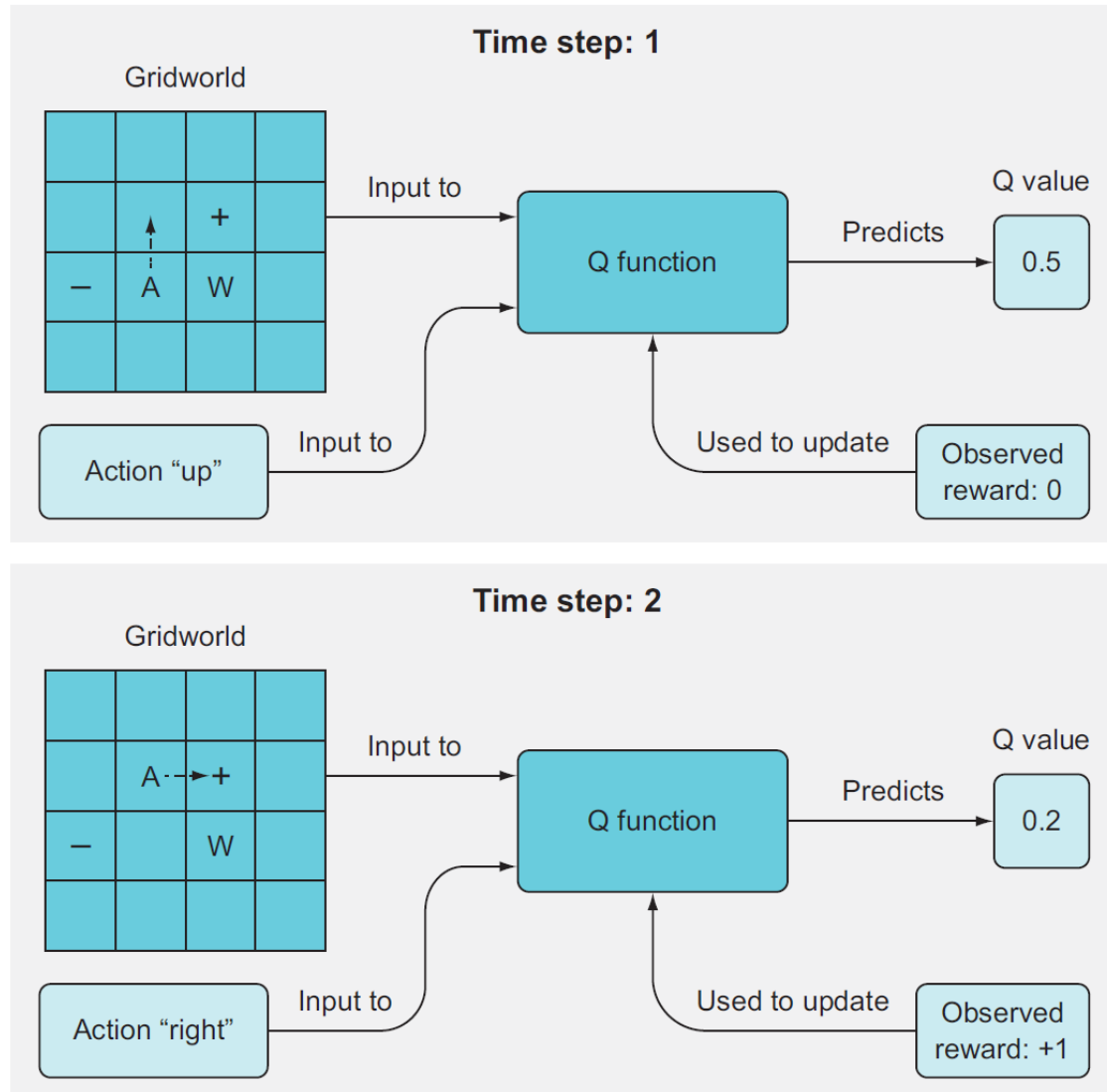
Source: Zai & Brown

A Deep RL Agent

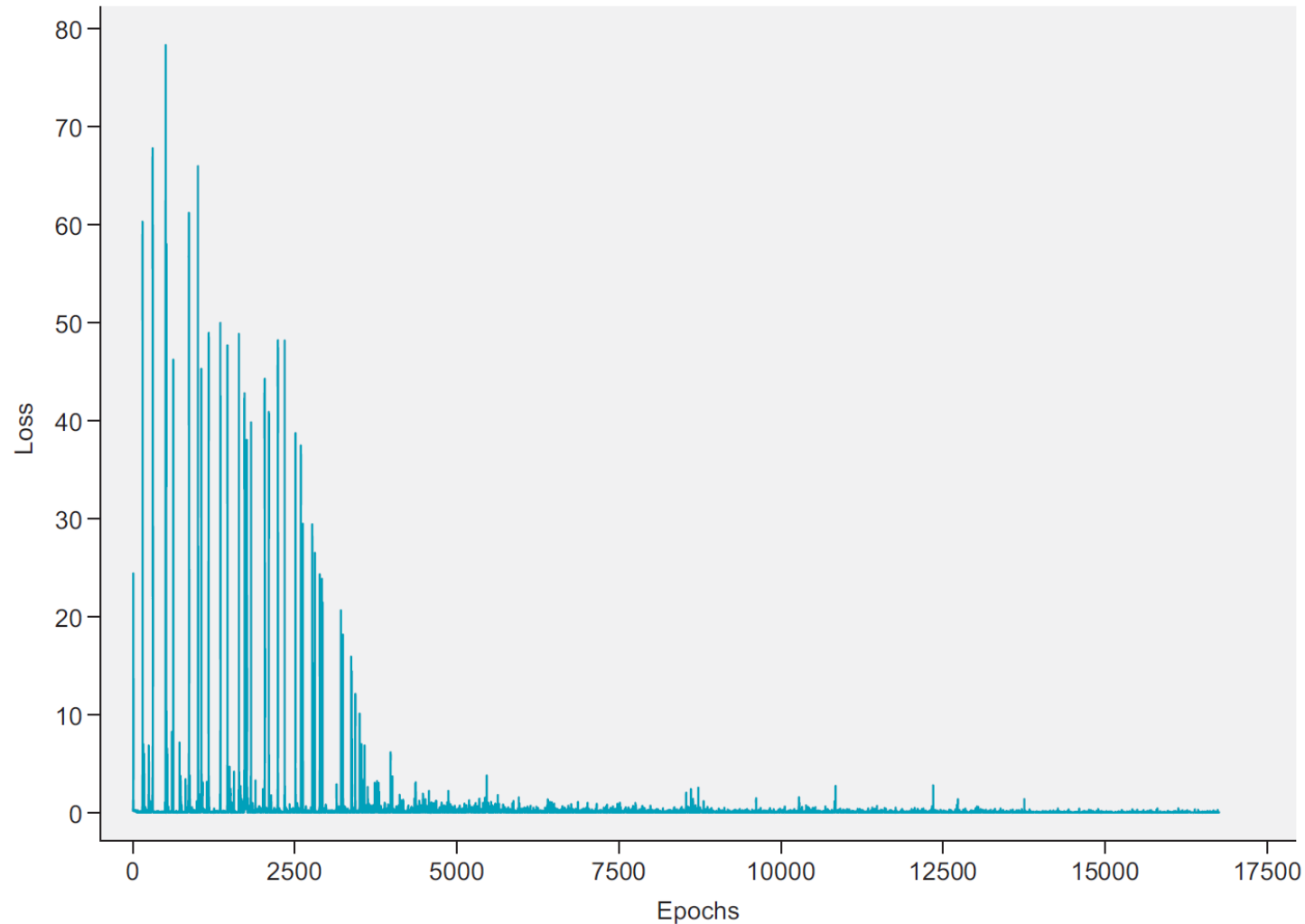


Source: Zai & Brown

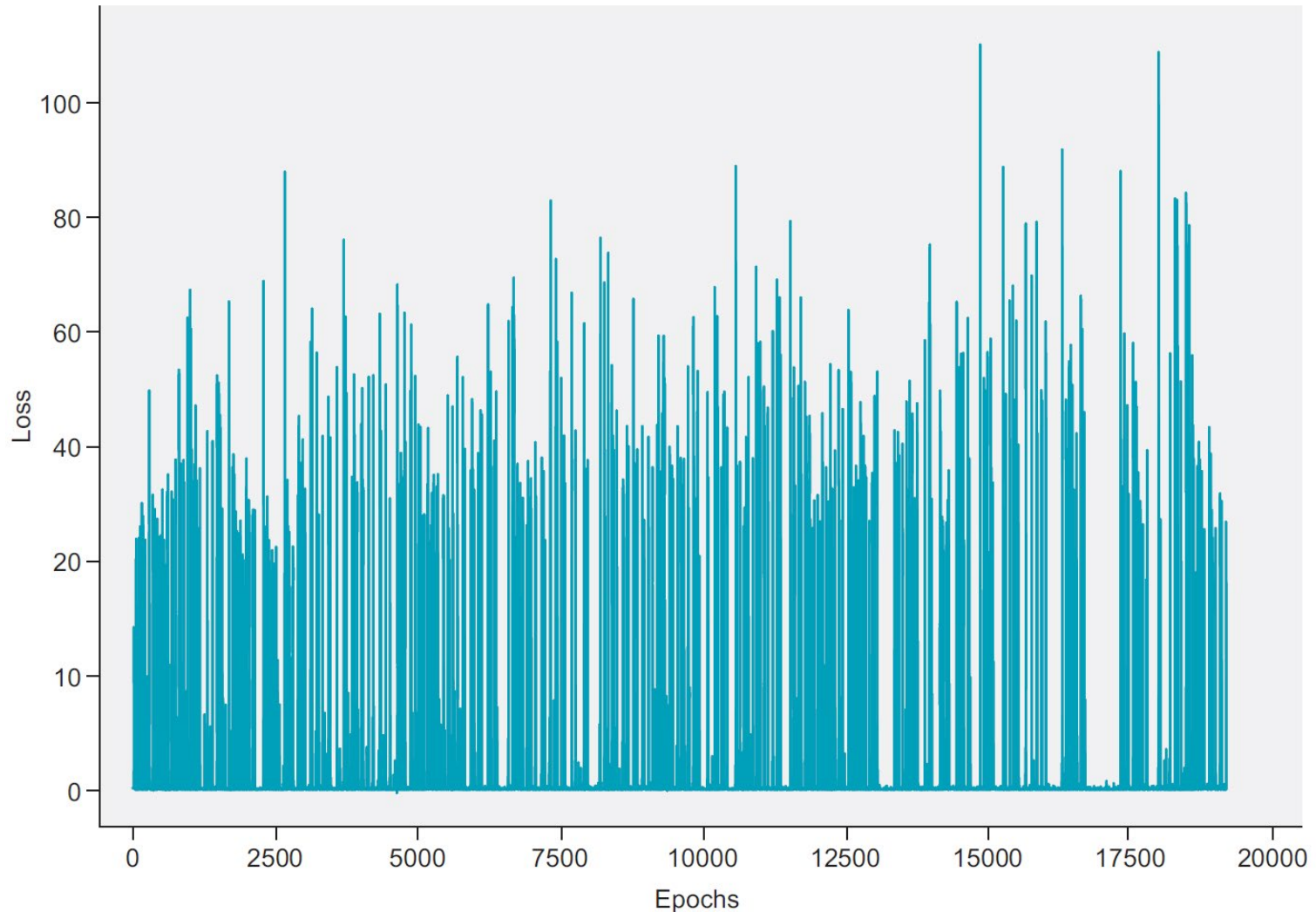
Deep Q Network



Loss as the Deep Q Net Learns on Fixed Goal Location



Loss if Trained on Changing Grid Configurations



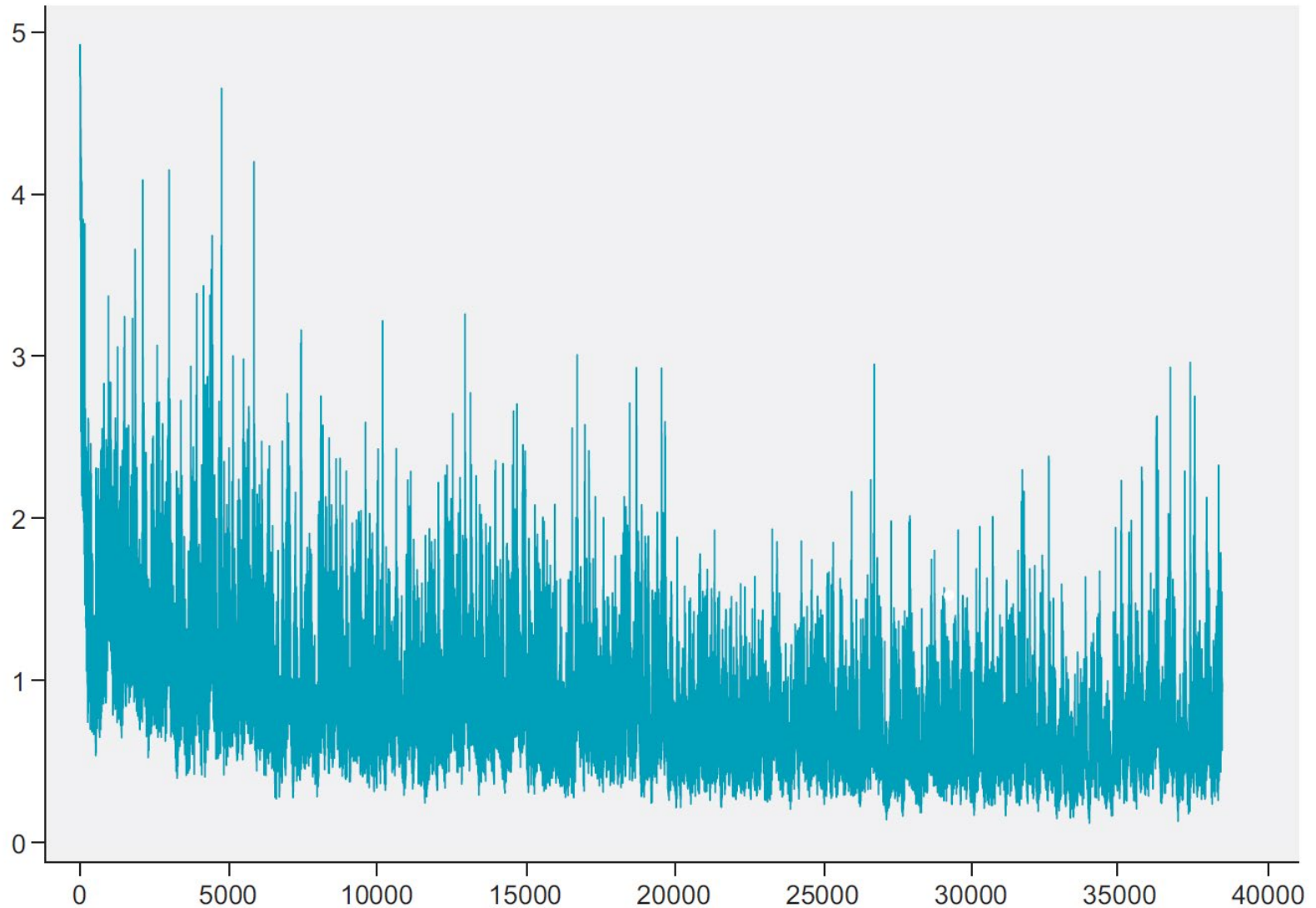
Two Problems with Value-Based Deep RL

- RL breaks two fundamental assumptions of supervised learning:
 1. Samples are not IID
 - Our samples come from a sequence, so they're correlated
 2. Non-stationary targets during training
 - The distribution of state-actions visited will change over time as the policy improves
 - The target state-action values will also change
 - The target state-action function changes radically when we change the location of the goal state

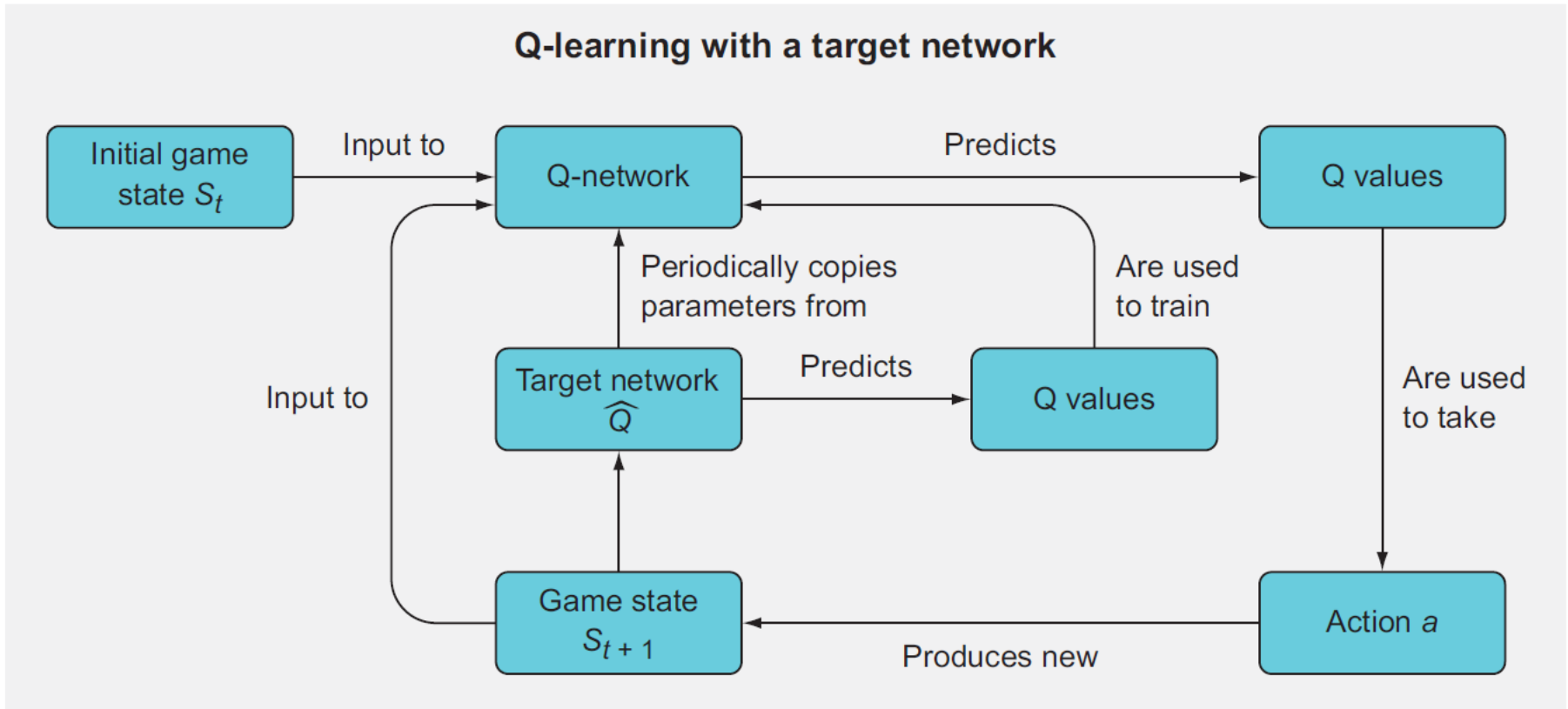
Catastrophic Forgetting and Experience Replay

- This is an issue with online sequential training in general
- Often the training set has no timestep feature to allow learning its sequential nature
- Switching tasks results in weights moving away from previously learned tasks to new tasks being learned, throwing away the previously-acquired knowledge
- Since the experiences are sequential, they are correlated which makes for inefficient learning
- Strategy: Experience Replay - Build up a buffer of experience, choose samples from the buffer at random, train in mini-batches

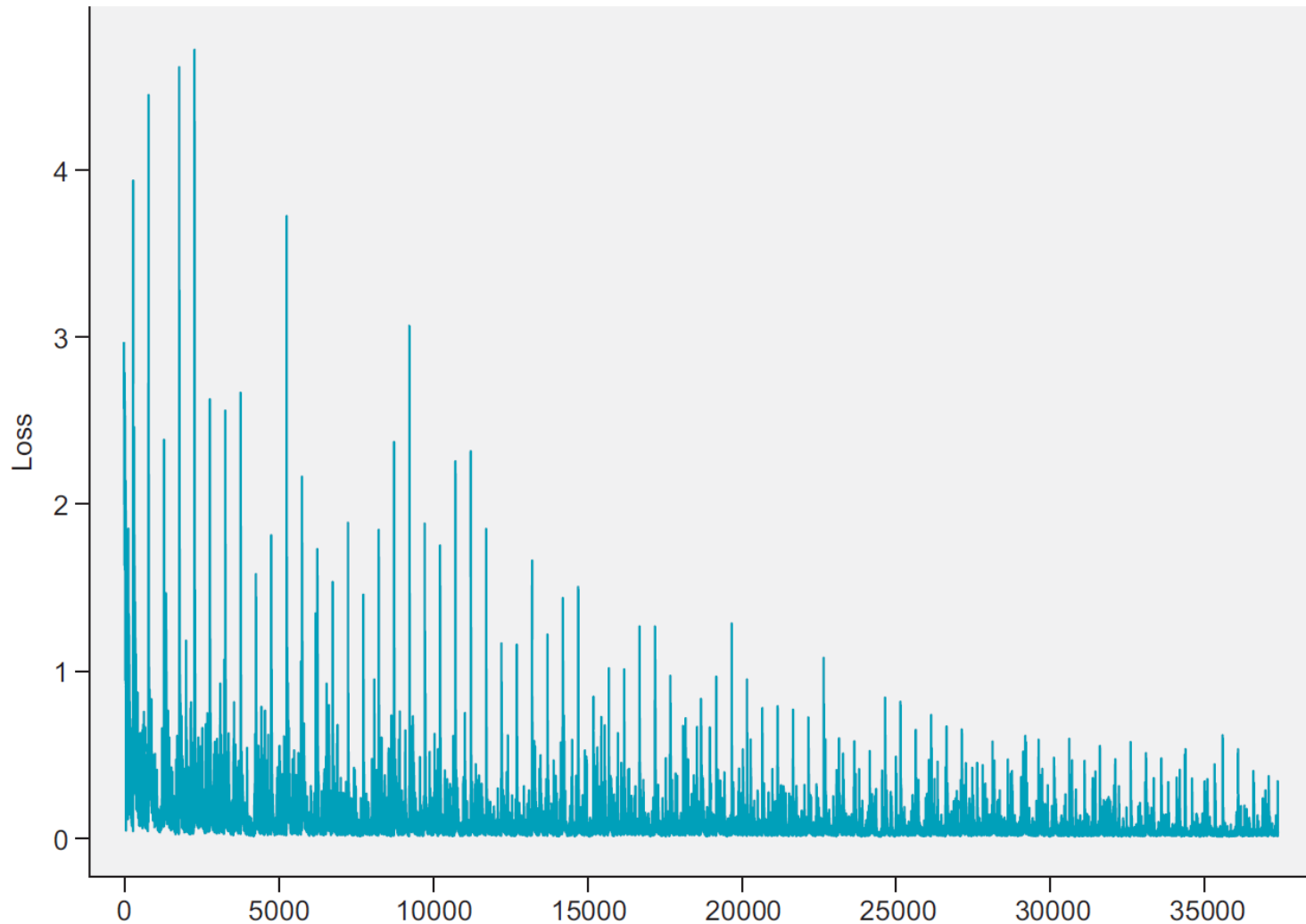
Loss Using Experience Replay



Improving Stability Using a Target Network



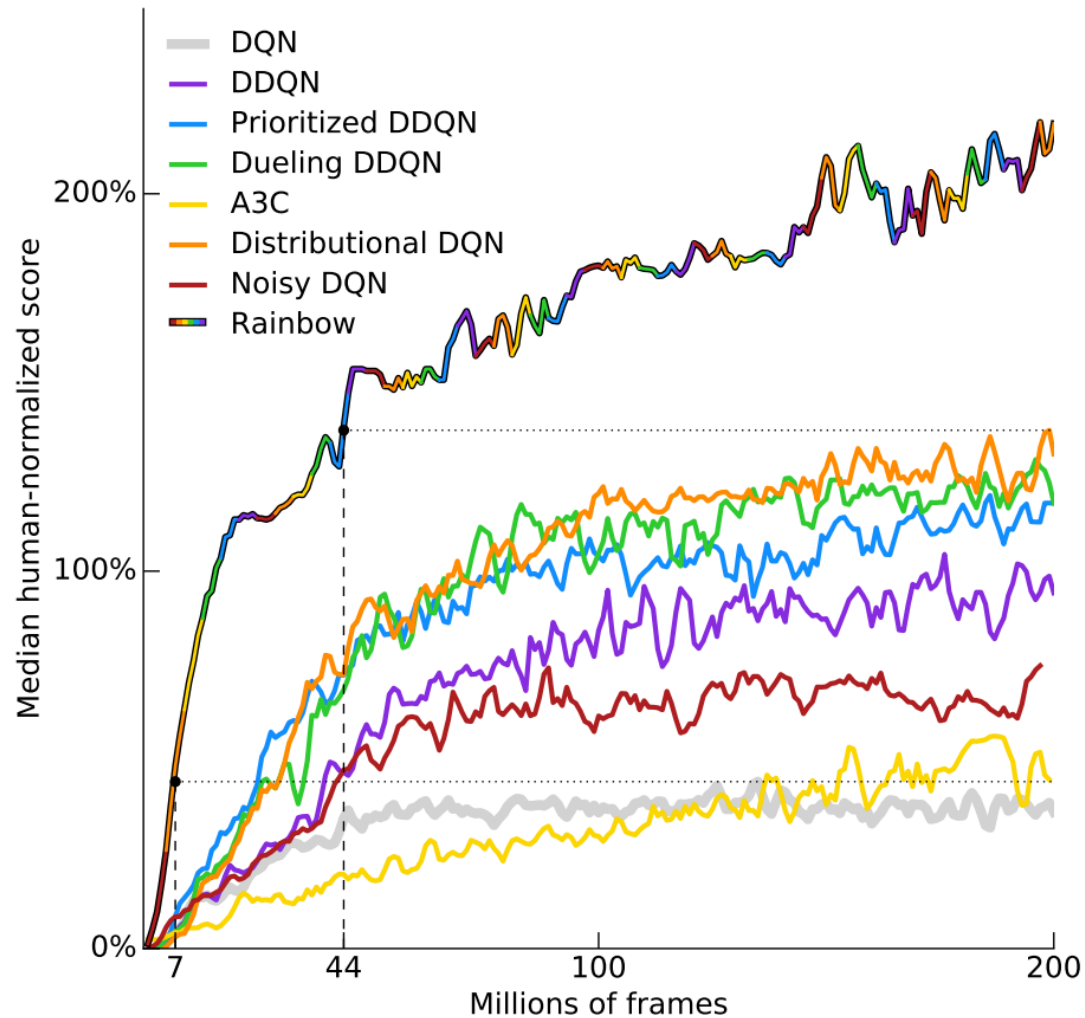
Loss Using Replay and Target Network



Rainbow DQN

- **Double Q-Learning:** Use two q estimators
- **Distributional RL:** Replace the q estimator (average reward) with an estimator of a probability distribution over rewards
- **Prioritized Experience Replay:** Prioritize experience buffer contents that the agent has explored less
- **Noisy Nets:** Adding a little noise to the neural net improves learning
- **Dueling Networks and A3C:** Use *advantage*
- See <https://arxiv.org/abs/1710.02298>

Rainbow DQN Performance





Module 4 – Section 3

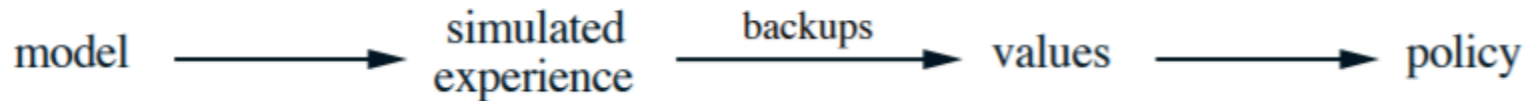
Integrated Planning and Learning

Planning and Learning

- Model-based methods
 - Dynamic Programming
 - Heuristic search
- Model-free methods
 - Monte Carlo
 - Temporal-Difference
- These approaches have a lot in common

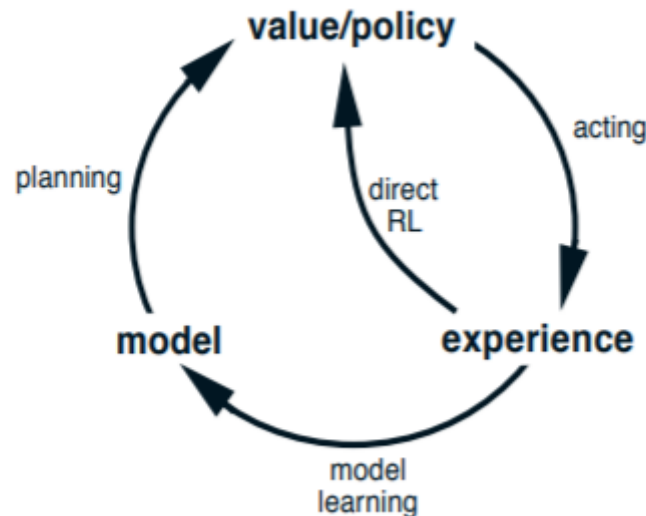
Models and Planning

- Model types
 - Distribution model: Assumed for Dynamic Programming
 - Sample models
- Models allow us to simulate experience
- In RL, the term **planning** means using a model to establish or improve a policy



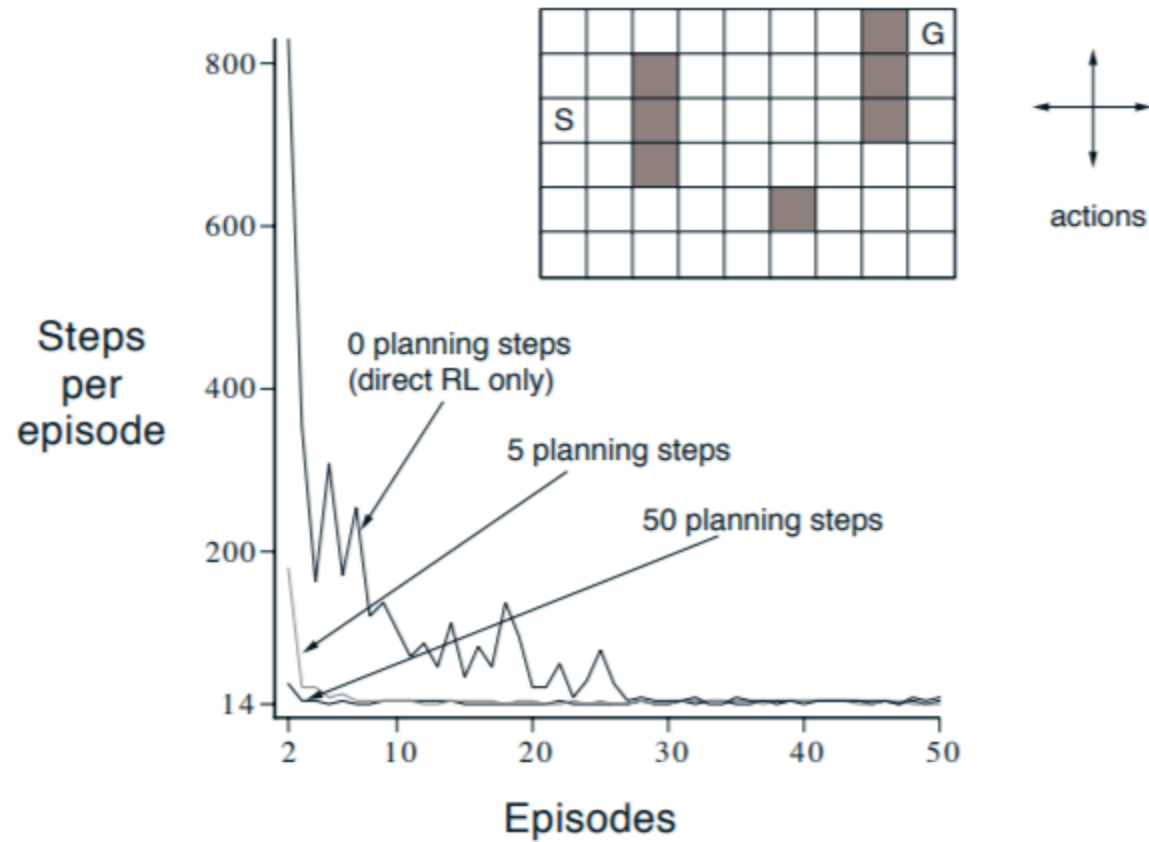
Dyna-Q (1 of 2)

- Why not learn $p(s, a)$ from experience if we don't have it?
- Build a table of experience: a list or distribution of (next state, reward) pairs observed for each (state, action) taken while acting in the real environment
- Then combine it with Q-learning



Source: Sutton & Barto

Dyna-Q (2 of 2)



Source: Sutton & Barto

Tabular Dyna-Q Algorithm

Tabular Dyna-Q

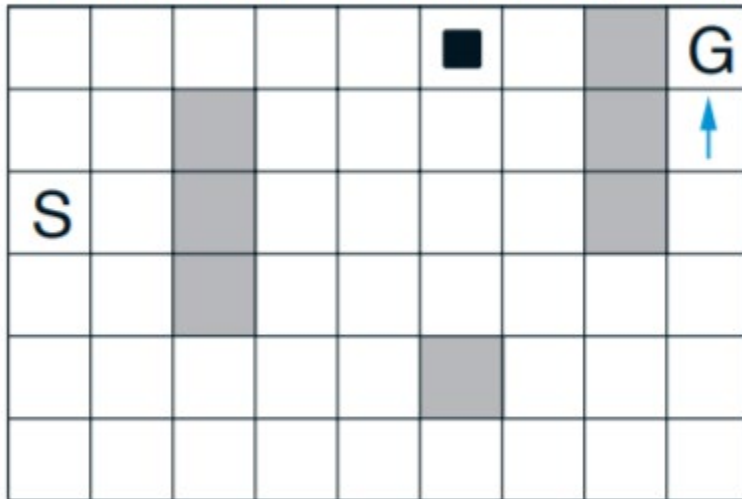
Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

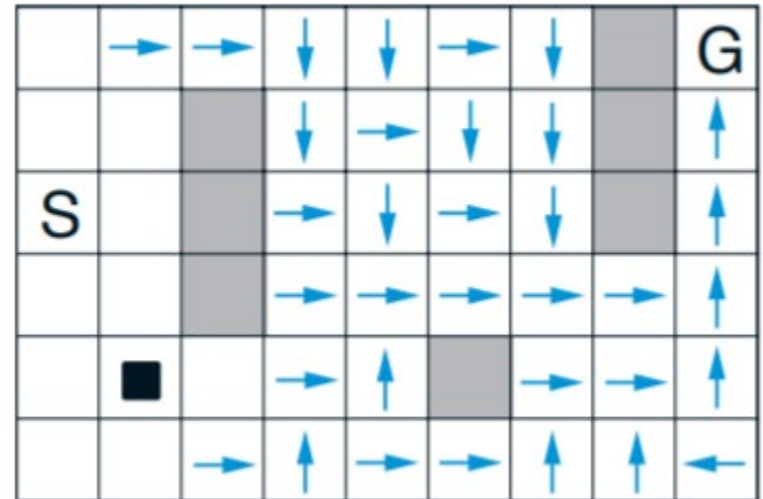
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

How Planning Accelerates Learning

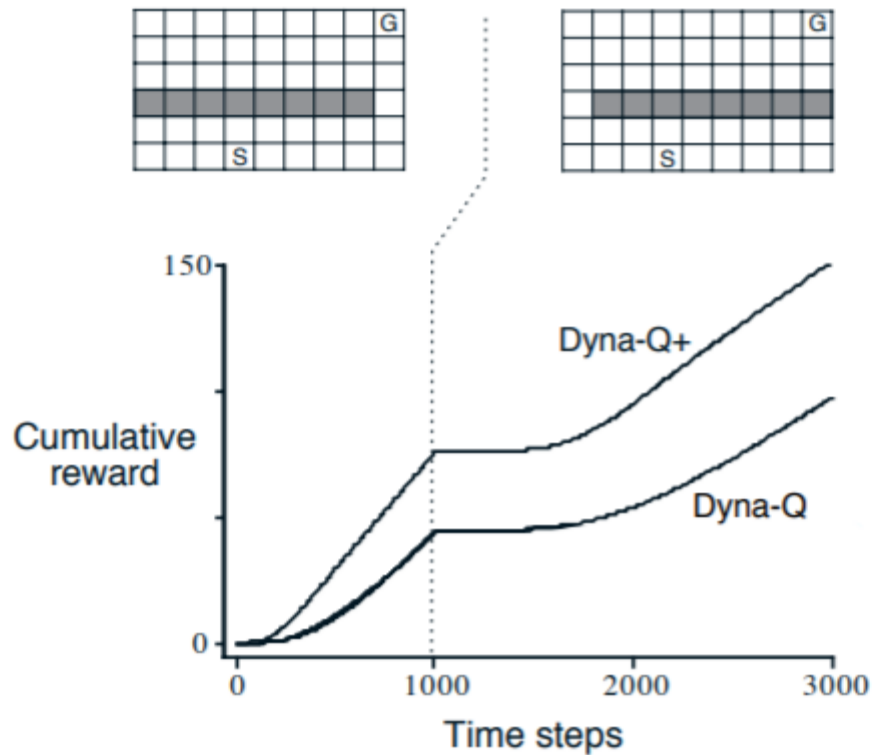
WITHOUT PLANNING ($n=0$)



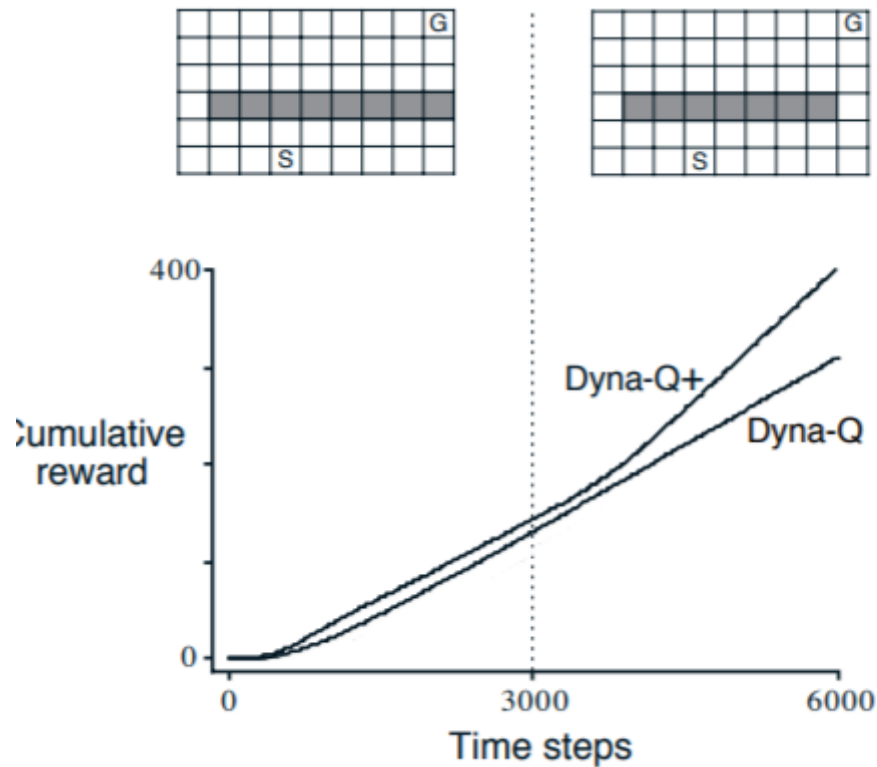
WITH PLANNING ($n=50$)



What if the Learned Model Is Now Wrong?



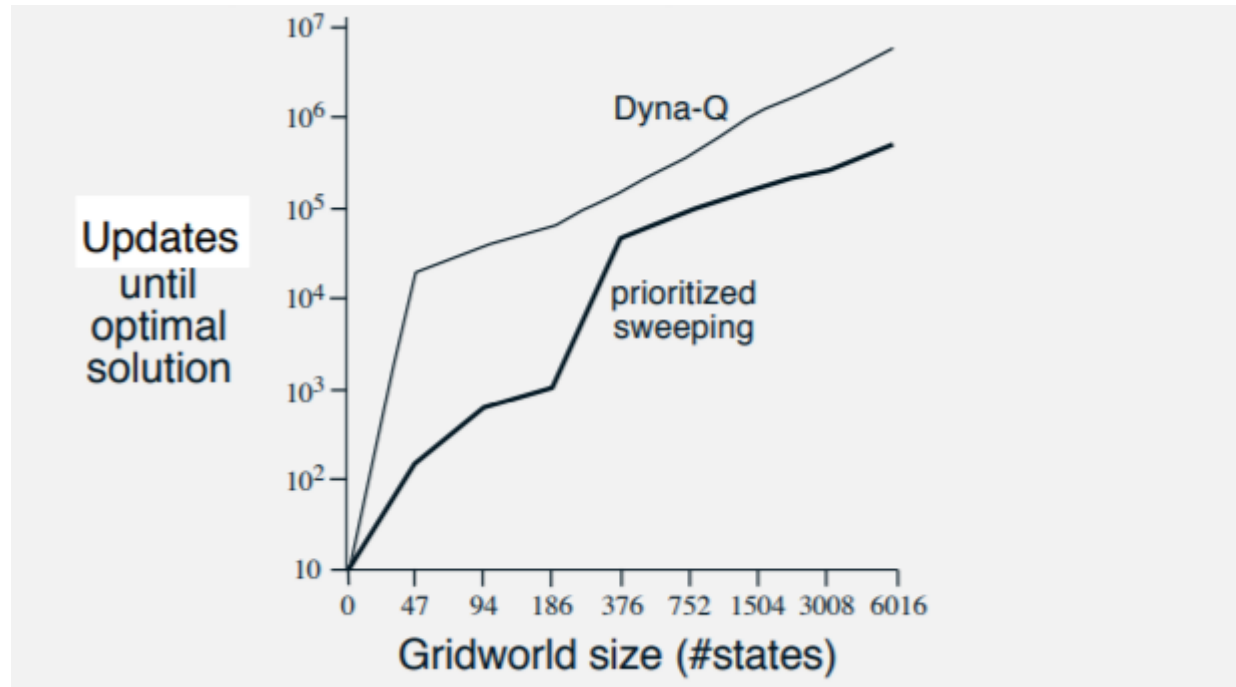
What if the Situation has Improved?



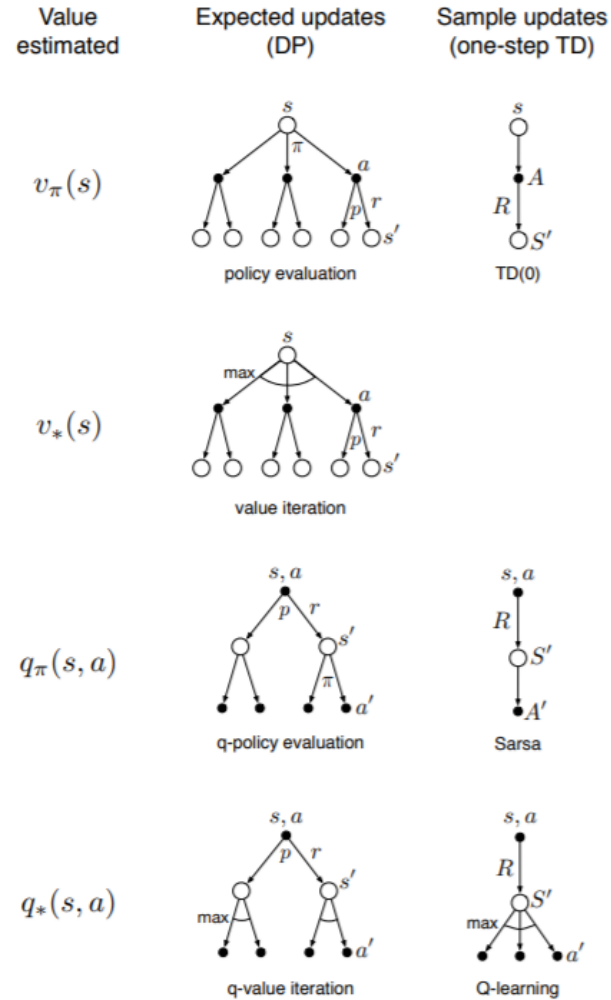
Prioritized Sweeping

- We'd like to focus our updates where they'll do the most good
- The notion here is to maintain an update priority queue based on the idea that state-actions whose Q have changed a lot are most likely to change π at the states leading into them

Prioritized Sweeping Improvement Over Dyna-Q



Expected vs. Sample Updates





Module 4 – Section 4

Resources and Wrap-up



Resources

- Sutton & Barto. Reinforcement Learning, 2nd Ed. MIT Press. 2018.
- Richard Sutton's website: <http://incompleteideas.net/>
- Pumperla & Ferguson. Deep Learning and the Game of Go, Chap. 10-14. Manning. 2019.
- Zai & Brown. Deep Reinforcement Learning in Action. Manning. 2020.
- Morales, Miguel. Grokking Deep Reinforcement Learning. Manning. 2020.
- Winder, Phil. Reinforcement Learning. O'Reilly. 2020.
- RainbowDQN: <https://arxiv.org/pdf/1710.02298.pdf>

Summary

- Deep Q Networks learn an approximation of the state-action function using deep neural nets
- Catastrophic forgetting can be a major issue where the model forgets what it previously learned if trained on small batches
- Experience Replay was developed to mitigate catastrophic forgetting and stabilize training
- A target network is a copy of the main deep Q network that also helps stabilize training the primary DQN
- We can accelerate learning of value functions by also learning a model of the environment

Next Week

- Policy Gradient methods: can we learn the optimal policy directly, without using value functions?
- Can we combine Policy Gradient with Value Function methods to create something even better?
- Spoiler: Yes and yes!



Any questions?



Thank You

Thank you for choosing the University of Toronto
School of Continuing Studies